

Owned Requirements: FR-009 to FR-011, FR-015 to FR-018

Owned Components: Category/Assignment Module, Search/Tracking Module, Database (MySQL)

Specialist Sections: 3.1 to 3.3, 3.5, 3.6, 4.1

3.1 Goals & Constraints

Goals: Support the complaint lifecycle (Submitted -> Assigned -> In Progress -> Resolved -> Closed). Meet NFR-001 (95% of normal requests within 10 seconds under the project test load). Keep modules separate and unit-testable (NFR-005, NFR-006). Enforce role-based access on every request (SEC-002, SEC-004).

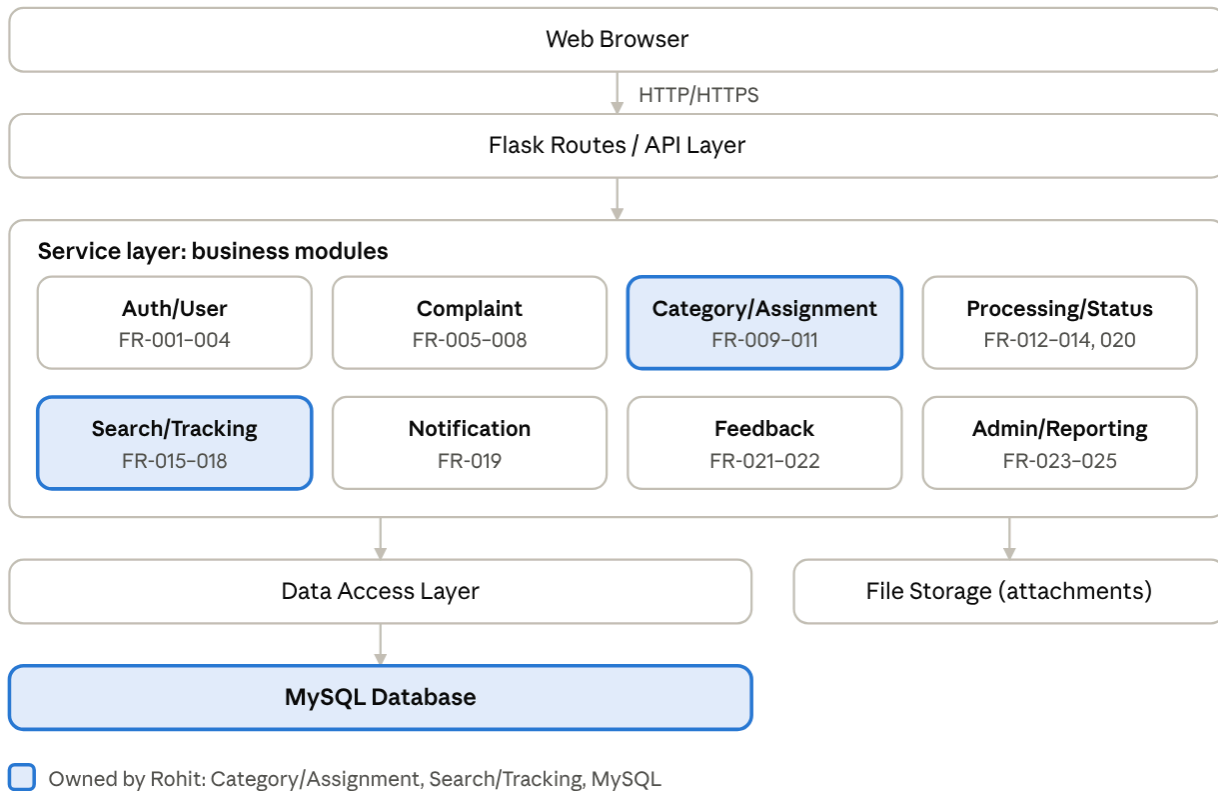
Constraints: Web application, Python Flask backend, MySQL database, Docker-based deployment, a single residential community, Agile development with Git/GitHub.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Resident	Easy tracking, privacy of own complaints, accurate status
Maintenance Staff	Clear view of assigned work, fast updates
Administrator	Correct assignment, category control, search and monitoring
Developers / Maintainers	Modularity, testability, consistent schema
Faculty / Evaluators	Traceability from SRS to design to tests

3.3 Component (UML) Diagram

All eight modules reach MySQL through one data access layer



SCMS component diagram · 8 modules, 1 database

Rohit's two modules and the database are highlighted; every module reaches MySQL through the data access layer.

3.4 Component Descriptions

3.4.1 Category/Assignment Module

The Category/Assignment Module maintains the predefined complaint categories and lets an Administrator create, edit and deactivate them (FR-009, FR-010). It lets an Administrator assign a complaint to a department or a specific maintenance staff member, and reassign it. The first assignment moves the complaint from Submitted to Assigned (FR-011). Every assignment is recorded and a notification event is raised.

3.4.2 Search/Tracking Module

The Search/Tracking Module lets a resident view the current status of their own complaints (FR-015) and exposes the status/action history kept for each complaint (FR-016). It provides search by Complaint ID, title and keyword (FR-017) and filtering by

category, status, department and date (FR-018). Results are always scoped to the role of the caller.

3.4.3 Database (MySQL)

The Database is the single relational store used by all modules. It holds the entities described in 3.6, enforces keys and constraints, and is accessed only through the data access layer.

3.5 Chosen Architecture Pattern and Rationale

Pattern: Layered architecture (modular monolith): Presentation (browser) -> API/route layer -> service layer (business rules per module) -> data access layer -> MySQL.

Rationale: The layers separate concerns and allow each service to be unit-tested without the full system (NFR-005, NFR-006). Role checks and the status workflow live in one place. A modular monolith deploys easily in Docker and suits a single community.

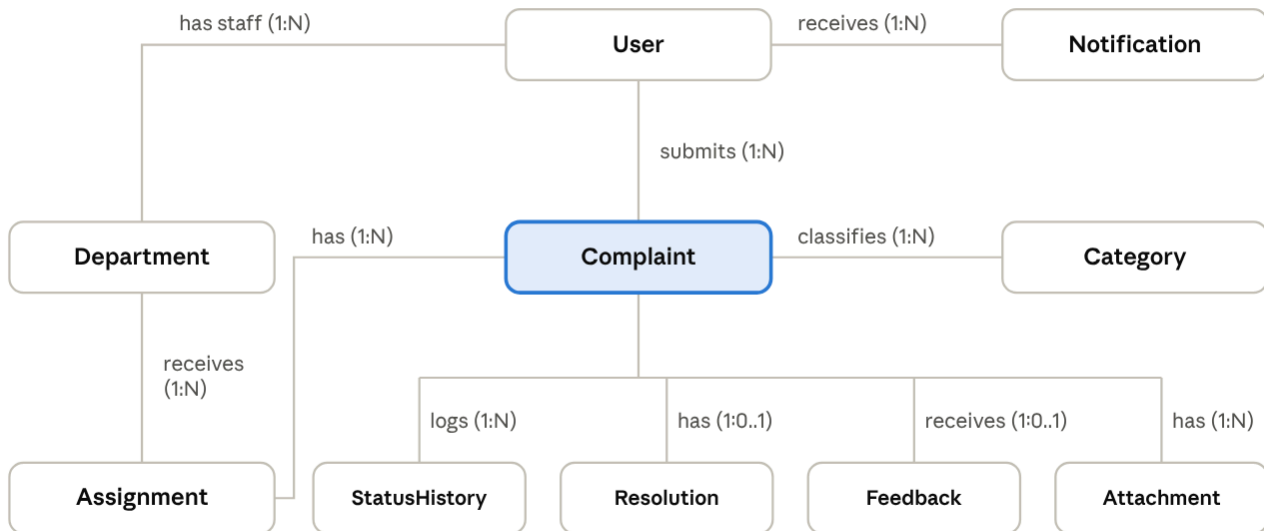
Rejected alternative: Microservices were rejected because the network, deployment and consistency overhead is not justified at this scale, for a four-person team working across six sprints.

3.6 Technology Stack & Data Stores

Python 3 / Flask (backend), HTML/CSS/JavaScript (frontend), MySQL (database), pytest (testing), Postman (API testing), Docker (deployment), Git/GitHub (version control), Jenkins (CI/CD), Draw.io/PlantUML (diagrams). Passwords are stored hashed and HTTPS is used in production.

ER Diagram

Complaint sits at the centre of the SCMS data model



SCMS ER diagram · 10 tables

Each complaint links to one resident, one category, and its assignments, status history, resolution, feedback and attachments.

Data Dictionary

All IDs are CHAR(36) UUIDs (matching SRS Appendix B). PK = primary key, FK = foreign key, NN = not null.

Table	Columns
User	user_id PK; name VARCHAR(100) NN; email VARCHAR(100) NN UNIQUE; password_hash VARCHAR(255) NN; contact_number VARCHAR(15) NN; unit_address VARCHAR(150) NN; role ENUM(Resident, Maintenance Staff, Administrator) NN; department_id FK to Department, NULL allowed
Department	department_id PK; name VARCHAR(100) NN UNIQUE; description VARCHAR(255)
Category	category_id PK; name VARCHAR(100) NN UNIQUE; description VARCHAR(255); is_active BOOLEAN NN default TRUE

Table	Columns
Complaint	complaint_id PK; resident_id FK to User NN; title VARCHAR(150) NN; category_id FK to Category NN; description TEXT (max 1000 chars) NN; location VARCHAR(200) NN; status ENUM(Submitted, Assigned, In Progress, Resolved, Closed) NN; created_at DATETIME NN; updated_at DATETIME NN
Attachment	attachment_id PK; complaint_id FK NN; file_path VARCHAR(255) NN; file_type VARCHAR(20); file_size_kb INT; uploaded_at DATETIME
Assignment	assignment_id PK; complaint_id FK NN; assigned_by FK to User NN; department_id FK NN; staff_id FK to User, NULL allowed; assigned_at DATETIME NN; is_current BOOLEAN NN
StatusHistory	history_id PK; complaint_id FK NN; old_status VARCHAR(20); new_status VARCHAR(20); changed_by FK to User NN; remarks VARCHAR(500); changed_at DATETIME NN
Resolution	resolution_id PK; complaint_id FK NN UNIQUE; resolved_by FK to User NN; details TEXT NN; resolved_at DATETIME NN
Notification	notification_id PK; user_id FK NN; complaint_id FK NN; message VARCHAR(255) NN; is_read BOOLEAN; created_at DATETIME NN
Feedback	feedback_id PK; complaint_id FK NN UNIQUE; resident_id FK NN; rating TINYINT NN (CHECK 1 to 5); comments VARCHAR(500); submitted_at DATETIME NN

3.8 Traceability to Requirements

The following rows cover the requirements owned by Rohit and map each requirement to its component and test cases.

Requirement ID	Component	Test Case IDs
FR-009	Category/Assignment Module	TC-ASG-01, TC-ASG-02

Requirement ID	Component	Test Case IDs
FR-010	Category/Assignment Module	TC-ASG-03, TC-ASG-04, TC-ASG-05
FR-011	Category/Assignment Module	TC-ASG-06, TC-ASG-07, TC-ASG-08
FR-015	Search/Tracking Module	TC-TRK-01, TC-TRK-02
FR-016	Search/Tracking Module	TC-TRK-03, TC-TRK-04
FR-017	Search/Tracking Module	TC-TRK-05, TC-TRK-06, TC-TRK-09
FR-018	Search/Tracking Module	TC-TRK-07, TC-TRK-08

4.1 Design Overview

SCMS is a layered, modular Flask application. Each module has routes, a service class holding its business rules, and repository functions that talk to MySQL. A shared authentication layer attaches the user's identity and role to each request. Role checks happen in the service layer, so rules such as "residents see only their own complaints" and "only Administrators assign" are enforced regardless of the UI. All status changes and assignments write to StatusHistory and Assignment, which preserves accountability.

4.3 API Design

The following are proposed design-level API interfaces for the Category/Assignment and Search/Tracking components. All endpoints require an authenticated session and return 401 when the user is not logged in. Errors use the format `{"error": {"code": "", "message": ""}}`.

Endpoint	Method	Role	Request	Success Response	Errors
/api/categories	GET	Any	none	200: list of category_id, name, description	401
/api/categories	POST	Admin	name, description	201: category_id, name	400, 403, 409 duplicate name
/api/categories/{id}	PUT	Admin	name, description, is_active	200: updated category	400, 403, 404, 409
/api/categories/{id}	DELETE	Admin	none	204	403, 404, 409 category in use
/api/departments	GET	Admin	none	200: list of department_id, name	403
/api/departments/{id}/staff	GET	Admin	none	200: list of user_id, name	403, 404

Endpoint	Method	Role	Request	Success Response	Errors
/api/complaints/{id}/assignment	POST	Admin	department_id, staff_id (optional)	200: complaint_id, status, department_id, staff_id, assigned_at	400, 403, 404, 409 complaint Resolved/Closed
/api/complaints/{id}/status	GET	Resident (own), Staff (assigned), Admin	none	200: complaint_id, status, updated_at	403, 404
/api/complaints/{id}/history	GET	Resident (own), Staff (assigned), Admin	none	200: list of old_status, new_status, changed_by, remarks, changed_at	403, 404
/api/complaints	GET	Role-scoped	query: q, category_id, status, department_id, from, to, page	200: items, page, total	400 invalid filter value